Complete Guide to LLM Fine-tuning

Qwopus3.5-27B LoRA Fine-tuning with Unsloth

Jackrong

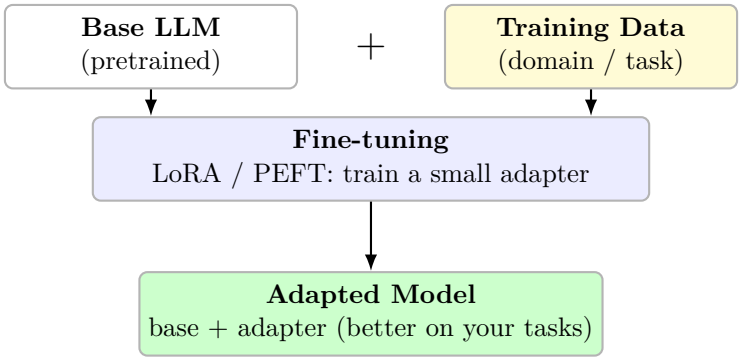
April 5, 2026

Abstract

This work proposes and implements a reasoning-oriented supervised fine-tuning pipeline for Qwen3.5-27B. Built with Unsloth on Google Colab, the workflow combines 4-bit quantized loading and rank-64 LoRA adaptation to enable efficient training of a 27B-scale model under single-GPU VRAM constraints. Unlike generic chat fine-tuning, the pipeline mixes three reasoning data sources and normalizes heterogeneous assistant outputs into a unified supervision format that combines `<think>...</think>` traces with final answers. Together with the Qwen thinking chat template and response-only supervision, the optimization target is explicitly focused on assistant-side reasoning continuation rather than memorizing full dialogue turns.

After training, the pipeline supports LoRA adapter checkpointing, merged 16-bit export, and multi-format GGUF release, forming a complete engineering chain from data normalization to parameter-efficient fine-tuning and deployment delivery. Importantly, although Qwen3.5 belongs to a multimodal model family, the notebook implemented in this study executes text-only reasoning SFT rather than vision-language joint training.

Keywords: Qwen3.5-27B, Reasoning SFT, LoRA, Unsloth, 4-bit Quantization, Google Colab, GGUF



LLM Fine-tuning: adapt a pretrained LLM to your domain with task-specific data (often by training a small LoRA adapter).

Author's Note — A Message to Builders

For beginners, hobbyists, and anyone curious about AI: this path is learnable.
The purpose of this document is not only to describe one training run, but also to communicate a broader message to beginners, hobbyists, and anyone curious about AI: fine-tuning, post-training, and even moderate-scale pretraining are not inaccessible technical rituals. They are engineering practices that can be learned, reproduced, and gradually mastered. With open-source models, public datasets, cloud compute platforms, and an increasingly mature training toolchain, what you often need is simply a Google account, a regular laptop, and sustained curiosity.
As a learner who also started from zero, I understand the uncertainty many newcomers face: environment setup complexity, opaque hyperparameters, and anxiety about compute resources often become the first barrier to entry. This is exactly why optimization toolchains such as Unsloth matter: by improving training efficiency and resource utilization, they substantially lower the practical threshold for large-model fine-tuning, turning what once required expensive hardware and specialized experience into something ordinary developers can attempt and master. In that sense, we all have the opportunity to stand on the shoulders of giants, understand models, adapt models, and give them new capabilities.

— **From one learner to another**

1 Prerequisites and Platform Overview

1.1 What is Unsloth?

Unsloth is a high-impact open-source team and toolkit for LLM post-training. Beyond efficient LoRA/QLoRA, it also supports full fine-tuning, continued pretraining, and low-precision setups (4-bit, 8-bit, 16-bit, FP8), while remaining compatible with Hugging Face/TRL workflows including SFT, GRPO, GSPO, and DPO.

Unsloth: quick summary

Unsloth's core advantage core advantage is systems-level optimization for post-training—official docs and benchmark pages repeatedly highlight around 2× training speedups and large VRAM savings (commonly reported near 70%, with RL workflows often claiming 50–90%), enabling longer context and larger-model fine-tuning on limited GPUs; on top of this, it still provides 250+ notebooks and broad support across 500+ models.

made with unsloth

1.2 What is Google Colab?

Google Colab is a cloud-hosted Jupyter notebook environment provided by Google. It lets you run Python code in the browser, connect to GPU runtimes, and quickly prototype machine learning workflows without configuring a local CUDA environment from scratch.

Why Colab is commonly used here

- Fast notebook-based experimentation for training and debugging.
- Easy integration with Google Drive for checkpoint persistence.
- Direct access to GPU-backed runtimes in Colab Pro/Pro+ tiers.

1.3 Hardware Requirement: NVIDIA CUDA GPU

For this project setup, you should use an **NVIDIA GPU with CUDA support**. CUDA is required by the PyTorch + accelerated training stack used in this guide, including 4-bit loading and LoRA fine-tuning workflows.

GPU requirement note

- Recommended: NVIDIA GPUs available in Colab (e.g., A100/H100 class when available).
- Training 27B-class models is not practical on CPU-only runtimes.
- If CUDA is unavailable, notebook cells related to model loading/training may fail.

Contents

1 Prerequisites and Platform Overview **2**
1.1 What is Unsloth? . . . 2
1.2 What is Google Colab? . . . 3
1.3 Hardware Requirement: NVIDIA CUDA GPU . . . 3

2 Introduction **6**
2.1 Background and Motivation . . . 6
2.2 Tech Stack Overview . . . 6

3 Environment Setup and Preparation **7**
3.1 Experiment Tracking (Optional) . . . 7
3.1.1 What it does . . . 7
3.2 Installing Dependencies . . . 8
3.2.1 Core libraries . . . 8

4 Model Loading and LoRA Configuration **9**
4.1 Loading the Pretrained Model . . . 9
4.1.1 How 4-bit quantization works . . . 9
4.1.2 Model loading code . . . 9
4.1.3 Best Practices: Thinking vs. Instruct Sampling Settings . . . 10
4.1.4 Template First: Confirm the Chat Format Before Training . . . 10
4.2 Configuring the LoRA Adapter . . . 11
4.2.1 LoRA principles . . . 11
4.2.2 LoRA configuration code . . . 11

5 Data Processing and Preprocessing **12**
5.1 Part I: Configuration and Multi-source Dataset Loading . . . 12
5.1.1 Pipeline goal . . . 12
5.2 Part II: Conversation Normalization and Reasoning-format Unification . . . 14
5.2.1 Normalization strategy . . . 14
5.3 Part III: Template Serialization, Length Control, and Format QA . . . 17
5.3.1 Final dataset construction . . . 17

6 Training Configuration and Execution **19**
6.1 Trainer Configuration . . . 19
6.1.1 Trainer code . . . 19
6.1.2 Key training parameters . . . 20
6.1.3 Detailed notes on key parameters . . . 20
6.2 Train on Responses Only (Recommended) . . . 20
6.3 Label Sanity Check (Before/After Training) . . . 21

7 Start Training (Most Important) **21**

8 Saving and Deployment **21**
8.1 Saving a 16-bit model . . . 21
8.2 Exporting GGUF (optional) . . . 22

9 Common Issues and Optimization Tips **23**
9.1 Typical failure modes . . . 23
9.2 Hyperparameter tuning workflow . . . 24

10 Summary and Outlook **24**
10.1 Key takeaways . . . 24
10.2 Typical metrics . . . 25
10.3 Next learning steps . . . 25
10.4 References . . . 26
10.5 Acknowledgements . . . 26

2 Introduction

Overview: Scope and goal

Project Goal

Goal. Adapt Qwen3.5-27B into a more structured, reasoning-efficient model by distilling high-quality analytical and step-by-step problem-solving patterns, with a focus on stronger performance in coding, mathematics, and offline analytical tasks while reducing unnecessarily long or repetitive reasoning on simpler queries.

Training Setting

Setting. Load Qwen3.5-27B through Unsloth in a memory-efficient format suitable for Colab, and perform response-only supervised fine-tuning on curated high-fidelity reasoning data to produce a more organized, transparent, and practically useful model for downstream inference and research-oriented experimentation.

2.1 Background and Motivation

Key Points: Why parameter-efficient fine-tuning (PEFT)?

With the rapid progress of Large Language Models (LLMs), efficiently adapting a general-purpose model to a specific domain or task has become a key challenge.

- Full-parameter fine-tuning is often expensive in compute and memory.
- PEFT methods (especially LoRA) train a small set of parameters while keeping the base model frozen, enabling fast iteration for individual developers and small teams.

2.2 Tech Stack Overview

Key stack only (focus points):

Component	Key point
Unsloth	Faster and more memory-efficient fine-tuning workflow for Colab.
LoRA (PEFT)	Train only low-rank adapters to reduce compute and cost.
4-bit quantization	Lower VRAM usage while keeping 27B-scale training practical.
Colab Pro GPU (A100/H100)	Provides the CUDA GPU runtime needed for this pipeline.

Table 1: Essential tech stack (condensed)

3 Environment Setup and Preparation

3.1 Experiment Tracking (Optional)

3.1.1 What it does

Weights & Biases (WandB) is a powerful experiment tracking platform. It can monitor training metrics in real time, including:

- Loss curves
- Learning-rate schedules
- VRAM utilization
- Training speed and ETA

Code: WandB login setup

```python
import os
import wandb
from google.colab import drive
from google.colab import userdata

drive.mount('/content/drive')
wandb_api_key = userdata.get('WANDB_API_KEY')
wandb.login(key=wandb_api_key)

drive_output_path = "/content/drive/MyDrive/Qwen3.5-27B--checkpoints"
os.makedirs(drive_output_path, exist_ok=True)
```

Warning: Colab usage notes

- Run this cell in Google Colab (not Kaggle), and grant Drive permission when prompted by `drive.mount()`.
- Store your WandB API key in Colab Secrets as `WANDB_API_KEY`; if login fails, re-check the secret name and value.
- Ensure your runtime has internet access and enough Drive space, otherwise checkpoint saving to `/content/drive/MyDrive/...` may fail.

3.2 Installing Dependencies

3.2.1 Core libraries

Fine-tuning an LLM typically requires these core libraries:

Library	Version	Purpose
Unsloth	GitHub (unsloth[base])	Installed from Unsloth GitHub for latest base runtime updates
PyTorch	2.8.0	Core deep learning backend used by this Colab setup
Transformers	5.2.0	Hugging Face model stack pinned to the notebook environment
TRL	0.22.2	Training utility stack aligned with Unsloth workflow
xformers	0.0.32.post2	Attention acceleration backend matched to the pinned torch version

Table 2: Core dependency list

Code: Install commands

```
%%capture
import os, importlib.util
!pip install --upgrade -qqq uv
if importlib.util.find_spec("torch") is None or "COLAB_" in "".join(os.environ.keys()):
    try: import numpy, PIL; _numpy = f"numpy==={numpy.__version__}"; _pil = f"pillow==={PIL.__version__}"
    except: _numpy = "numpy"; _pil = "pillow"
    !uv pip install -qqq \
        "torch==2.8.0" "triton>=3.3.0" {_numpy} {_pil} torchvision bitsandbytes xformers==0.0.32.post2 \
        "unsloth_zoo[base] @ git+https://github.com/unslothai/unsloth-zoo" \
        "unsloth[base] @ git+https://github.com/unslothai/unsloth"
elif importlib.util.find_spec("unsloth") is None:
    !uv pip install -qqq unsloth
!uv pip install --upgrade --no-deps tokenizers trl==0.22.2 unsloth unsloth_zoo
!uv pip install transformers==5.2.0
# causal_conv1d is supported only on torch==2.8.0. If you have newer torch versions, please wait 10 minutes!
!uv pip install --no-build-isolation flash-linear-attention causal_conv1d==1.6.0
```

Tip: Installation notes

- Use the `%%capture` magic to hide install logs and keep the notebook clean.
- Installation usually takes about 3–5 minutes.
- If you see version warnings but Unsloth imports correctly, you can typically ignore them.

4 Model Loading and LoRA Configuration

4.1 Loading the Pretrained Model

4.1.1 How 4-bit quantization works

4-bit quantization stores model parameters in 4-bit precision, dramatically reducing VRAM usage while keeping inference and training stable (especially when combined with quantization-aware kernels and mixed precision). In practice, 4-bit is often the best balance for single-GPU fine-tuning.

4.1.2 Model loading code

Code: Load Qwen3.5-27B with Unsloth

```python
from unsloth import FastLanguageModel
import torch

fourbit_models = [
    "unsloth/Qwen3-4B-Instruct-2507-unsloth-bnb-4bit", # Qwen 14B 2x faster
    "unsloth/Qwen3-4B-Thinking-2507-unsloth-bnb-4bit",
    "unsloth/Qwen3-8B-unsloth-bnb-4bit",
    "unsloth/Qwen3-14B-unsloth-bnb-4bit",
    "unsloth/Qwen3-32B-unsloth-bnb-4bit",

    # 4bit dynamic quants for superior accuracy and low memory use
    "unsloth/gemma-3-12b-it-unsloth-bnb-4bit",
    "unsloth/Phi-4",
    "unsloth/Llama-3.1-8B",
    "unsloth/Llama-3.2-3B",
    "unsloth/orpheus-3b-0.1-ft-unsloth-bnb-4bit" # [NEW] We support TTS models!
] # More models at https://huggingface.co/unsloth

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/Qwen3.5-27B",
    max_seq_length = 32768, # Choose any for long context!
    load_in_4bit = True,  # 4 bit quantization to reduce memory
    load_in_8bit = False, # [NEW!] A bit more accurate, uses 2x memory
    full_finetuning = False, # [NEW!] We have full finetuning now!
)
```

Warning: Common pitfalls

- If you hit OOM, reduce `max_seq_length` (e.g., 8192 or 16384) or use smaller batch size.
- Make sure the model identifier matches an existing HF repo name.
- If the model is gated, you must provide a valid HF token.

4.1.3 Best Practices: Thinking vs. Instruct Sampling Settings

Qwen models are commonly used in two inference modes: **Instruct** (instruction-following chat) and **Thinking** (reasoning with explicit thinking tags). The recommended sampling settings are different: Thinking typically uses a lower temperature and a higher `top_p` to improve reasoning stability.

Parameter	Instruct (Qwen3)	Thinking (Qwen3/Qwen3.5)
Temperature	0.7	0.6/0.7/1
Top P	0.80	0.95
TopK	20	20

Table 3: Recommended sampling settings for Qwen Instruct vs. Thinking (rule-of-thumb)

Output length: For most queries, set the maximum output length to 32768 tokens (if your runtime/VRAM allows); this is sufficient for many long-form responses.

4.1.4 Template First: Confirm the Chat Format Before Training

Why this matters before SFT

If you have already chosen the model to train, the next critical step is to confirm its corresponding chat template. Different model families and companies use different role markers, separators, special tokens, and system/user/assistant formatting rules.
If your training data format does not match the model's native chat template, you may see unstable loss, weaker instruction-following behavior, or degraded response quality. In short: **pick the model first, then align the dataset and prompt formatting to that model's template**. The example below shows a typical template structure.

Code: Qwen3 Chat template Example Main Part (Instruct / Thinking)

```
<|im_start|>user
Hey there! What is 1+1? <|im_end|>
<|im_start|>assistant
<think>

# Instruct mode: <think></think>answer
# Thinking mode: may include <think>..reasoning..</think>answer

</think>
The answer is 2.
<|im_end|>
```

4.2 Configuring the LoRA Adapter

4.2.1 LoRA principles

LoRA (Low-Rank Adaptation) is a parameter-efficient fine-tuning method. Instead of updating all original model weights, LoRA keeps the pretrained model frozen and adds two small trainable low-rank matrices (often denoted as A and B) to selected linear layers. The product of these matrices learns a lightweight task-specific update, so the model can adapt to new domains with far fewer trainable parameters. In practice, this greatly reduces VRAM usage and training cost while preserving most of the base model's capability, which is why LoRA is widely used for single-GPU or Colab-based LLM fine-tuning.

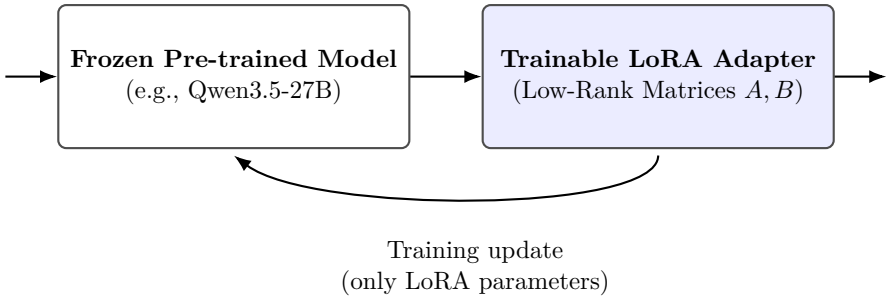


Figure 1: LoRA (Low-Rank Adaptation) principle: keep the base model frozen and learn a small, trainable low-rank update.

4.2.2 LoRA configuration code

Code: Attach LoRA adapters

```
model = FastLanguageModel.get_peft_model(
    model,
    r = 64, # Choose any number > 0 ! Suggested 8, 16, 32, 64, 128
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj",
                      "out_proj",],
    lora_alpha = 64,
    lora_dropout = 0, # Supports any, but = 0 is optimized
    bias = "none",    # Supports any, but = "none" is optimized
    # [NEW] "unsloth" uses 30% less VRAM, fits 2x larger batch sizes!
    use_gradient_checkpointing = "unsloth", # True or "unsloth" for very long context
    random_state = 3407,
    use_rslora = False,  # We support rank stabilized LoRA
    loftq_config = None, # And LoftQ
)
```

Tip: Recommended starting values

- `r`: 8/16/64/128 (higher rank can improve quality but costs more VRAM).
- `lora_alpha`: often set to `r` or `2r` (twice r) as a practical baseline.
- `target_modules`: start with attention + MLP projections (as above).

5 Data Processing and Preprocessing

5.1 Part I: Configuration and Multi-source Dataset Loading

5.1.1 Pipeline goal

This stage defines the data pipeline configuration and loads three heterogeneous reasoning datasets into a unified workflow. The key design choice is to sample each source with a fixed ratio so that high-quality reasoning traces and conversational diversity are both preserved.

Core principle for SFT beginners

Dataset selection largely determines what the model finally learns. Beginners often assume model size is the most important factor, but in supervised fine-tuning, dataset design usually has a larger impact than simply running more steps. In practice, these factors are often the strongest drivers of final quality:

- **Data quality** (signal-to-noise, correctness, and reasoning faithfulness).
- **Data style** (answer tone, structure, verbosity, and formatting habits).
- **Task distribution** (which capabilities are emphasized most often).
- **Format consistency** (stable template/role formatting across samples).

Bottom line: for SFT, these data decisions usually matter more than "just training a bit longer."

- Set global controls (seed, context window, per-dataset sample sizes).
- Initialize the Qwen thinking chat template early for downstream serialization.
- Load three sources with fallback logic for schema-specific dataset issues.

Note. The dataset choices and hyperparameters shown below are not the final production training settings. In real projects, you should tune them based on your training goals, dataset characteristics, hardware, and runtime environment. This section is provided as an example to help you understand the pipeline and workflow. Code can be generated or assisted by AI, but you still need to understand what each step is doing and why.

Detailed reference: TRL SFTTrainer

For a full reference of the supervised fine-tuning trainer, see: https://huggingface.co/docs/trl/sft_trainer

What this page explains in detail:

- How to use `SFTTrainer` for quick-start SFT training.
- Supported dataset schemas (language modeling and prompt-completion; standard and conversational formats).
- Core SFT mechanics: preprocessing, tokenization, next-token cross-entropy loss, label shifting, and padding-mask handling.
- Practical options: packing, `assistant_only_loss`, `completion_only_loss`, and PEFT/LoRA integration.
- Common training metrics and key customization options in `SFTConfig`.

Code: Configuration and dataset loading

```python
from datasets import load_dataset, concatenate_datasets, Dataset
from unsloth.chat_templates import get_chat_template
import re
import json
import multiprocessing as mp
import pandas as pd

RANDOM_SEED = 12181531
MAX_CONTEXT_WINDOW = 8192

num_samples_dict = {
    "ds1": 3900,  # nohurry/Opus-4.6-Reasoning-3000x-filtered
    "ds2": 700,   # Jackrong/Qwen3.5-reasoning-700x
    "ds3": 9633,  # Roman1111111/claude-opus-4.6-10000x
}

tokenizer = get_chat_template(
    tokenizer,
    chat_template="qwen3-thinking",
)

def load_ds3_via_pandas_parquet():
    parquet_path = (
        "hf://datasets/Roman1111111/claude-opus-4.6-10000x"
        "@refs/convert/parquet/default/train/0000.parquet"
    )
    df = pd.read_parquet(parquet_path)
    return Dataset.from_pandas(df, preserve_index=False)

def load_and_sample(dataset_name, sample_count=None, split="train", subset=None):
    try:
```

```
        if subset:
            ds = load_dataset(dataset_name, subset, split=split)
        else:
            ds = load_dataset(dataset_name, split=split)
    except ValueError as e:
        err = str(e)
        if dataset_name == "Roman1111111/claude-opus-4.6-10000x" and "Feature type 'Json' not found" in err:
            ds = load_ds3_via_pandas_parquet()
        else:
            raise

    if sample_count is not None:
        sample_count = min(sample_count, len(ds))
        ds = ds.shuffle(seed=RANDOM_SEED).select(range(sample_count))

    return ds

# ds1: problem / thinking / solution
# ds2: multi-turn conversation
# ds3: messages with possible reasoning fields
ds1 = load_and_sample("nohurry/Opus-4.6-Reasoning-3000x-filtered", num_samples_dict["ds1"], split="train")
ds2 = load_and_sample("Jackrong/Qwen3.5-reasoning-700x", num_samples_dict["ds2"], split="train")
ds3 = load_and_sample("Roman1111111/claude-opus-4.6-10000x", num_samples_dict["ds3"], split="train")
```

5.2 Part II: Conversation Normalization and Reasoning-format Unification

5.2.1 Normalization strategy

Because the three datasets use different schemas (problem/solution pairs, multi-turn chat, and message objects with optional reasoning fields), we normalize them into a single `conversations` structure and enforce an assistant output format centered on `<think>...</think>` plus final answer text.

- Standardize assistant messages into a reasoning-visible format.
- Parse mixed message types (dict / JSON string) safely.
- Convert each source with dataset-specific formatting functions.

Code: Multi-source formatting to unified conversations

```python
def _strip(x):
    return (x or "").strip()

THINK_BLOCK_RE = re.compile(r"<think>.*?</think>", flags=re.DOTALL)

def normalize_assistant_to_think_solution(text: str) -> str:
    text = _strip(text)
    if not text:
        return "<think></think>\n"

    m = THINK_BLOCK_RE.search(text)
    if m:
        think_block = m.group(0).strip()
        rest = text[m.end():].lstrip()
        return f"{think_block}\n{rest}".rstrip() if rest else f"{think_block}\n"

    return f"<think></think>\n{text}".rstrip()

def build_assistant_with_reasoning(content: str, reasoning: str = "") -> str:
    content = _strip(content)
    reasoning = _strip(reasoning)

    if "<think>" in content and "</think>" in content:
        return normalize_assistant_to_think_solution(content)

    if reasoning:
        return f"<think>{reasoning}</think>\n{content}" if content else f"<think>{reasoning}</think>\n"

    return normalize_assistant_to_think_solution(content)

def parse_message_item(m):
    if isinstance(m, dict):
        return m
    if isinstance(m, str):
        s = m.strip()
        if not s:
            return None
        try:
            obj = json.loads(s)
            return obj if isinstance(obj, dict) else None
        except Exception:
            return None
    return None

def format_ds1(examples):
    out = []
    for p, t, s in zip(examples.get("problem", []), examples.get("thinking", []), examples.get("solution", [])):
        p, t, s = _strip(p), _strip(t), _strip(s)
        if not p or not s:
```

```
            continue
        assistant = f"<think>{t}</think>\n{s}" if t else f"<think></think>\n{s}"
        out.append([
            {"role": "user", "content": p},
            {"role": "assistant", "content": assistant},
        ])
    return {"conversations": out}

def format_ds2(examples):
    out = []
    for conv in examples.get("conversation", []):
        if not conv:
            continue
        cleaned = []
        for m in conv:
            frm = (m.get("from") or "").strip()
            val = m.get("value", "")
            if frm == "human":
                cleaned.append({"role": "user", "content": _strip(val)})
            elif frm == "gpt":
                cleaned.append({"role": "assistant", "content": normalize_assistant_to_think_solution(val)})
        if len(cleaned) >= 2 and cleaned[-1]["role"] == "assistant":
            out.append(cleaned)
    return {"conversations": out}

def format_ds3(examples):
    out = []
    for msgs in examples.get("messages", []):
        if not msgs:
            continue
        parsed = [pm for pm in (parse_message_item(m) for m in msgs) if pm is not None]
        if not parsed:
            continue

        convo = [m for m in parsed if m.get("role") != "system"]
        if len(convo) < 2 or convo[-1].get("role") != "assistant":
            continue

        cleaned = []
        for m in convo:
            role = m.get("role")
            content = m.get("content", "")
            reasoning = m.get("reasoning", "")
            if role == "assistant":
                content = build_assistant_with_reasoning(content, reasoning)
            else:
                content = _strip(content)
            if role in ("user", "assistant") and content is not None:
                cleaned.append({"role": role, "content": content})

```

```
        if len(cleaned) >= 2 and cleaned[-1]["role"] == "assistant":
            out.append(cleaned)

    return {"conversations": out}

ds1 = ds1.map(format_ds1, batched=True, remove_columns=ds1.column_names)
ds2 = ds2.map(format_ds2, batched=True, remove_columns=ds2.column_names)
ds3 = ds3.map(format_ds3, batched=True, remove_columns=ds3.column_names)
```

5.3 Part III: Template Serialization, Length Control, and Format QA

5.3.1 Final dataset construction

After normalization, we merge the three datasets, serialize each conversation through the Qwen thinking template, filter over-length samples by tokenized length, and run a final assistant-format quality check before training.

- Keep only non-empty normalized conversations.
- Build training text via `apply_chat_template`.
- Enforce context-window constraints and reasoning tag integrity.

Code: Merge, template, filter, and validate

```
ds1 = ds1.filter(lambda x: x["conversations"] is not None and len(x["conversations"])
    > 0)
ds2 = ds2.filter(lambda x: x["conversations"] is not None and len(x["conversations"])
    > 0)
ds3 = ds3.filter(lambda x: x["conversations"] is not None and len(x["conversations"])
    > 0)

combined_dataset = concatenate_datasets([ds1, ds2, ds3]).shuffle(seed=RANDOM_SEED)

def formatting_prompts_func(examples):
    convos = examples["conversations"]
    texts = [
        tokenizer.apply_chat_template(
            convo,
            tokenize=False,
            add_generation_prompt=False,
        )
        for convo in convos
    ]
    return {"text": texts}

dataset = combined_dataset.map(formatting_prompts_func, batched=True)

num_proc = mp.cpu_count()
```

```python
_text_tok = getattr(tokenizer, "tokenizer", tokenizer)

def filter_long_sequences_batched(examples):
    texts = examples["text"]
    tokenized = _text_tok(
        texts,
        truncation=False,
        padding=False,
        add_special_tokens=False,
    )["input_ids"]
    return [len(toks) <= MAX_CONTEXT_WINDOW for toks in tokenized]

dataset = dataset.filter(filter_long_sequences_batched, batched=True, num_proc=
    num_proc)

def check_assistant_format(examples):
    convos = examples["conversations"]
    ok = []
    for convo in convos:
        good = True
        for m in convo:
            if m["role"] == "assistant":
                c = m.get("content", "")
                if "<think>" not in c or "</think>" not in c:
                    good = False
                    break
                if not re.search(r"</think>\n", c):
                    good = False
                    break
        ok.append(good)
    return {"_ok": ok}

check = dataset.map(
    check_assistant_format,
    batched=True,
    remove_columns=dataset.column_names,
    num_proc=num_proc,
)

bad = len(check) - sum(check["_ok"])
if bad > 0:
    dataset = dataset.filter(lambda x: all(
        (m["role"] != "assistant") or (
            ("<think>" in m["content"]) and ("</think>\n" in m["content"])
        )
        for m in x["conversations"]
    ))

print(dataset[0]["text"][:8000])
```

Tip: Practical checks before training

- Print a few post-template samples to verify role separators and reasoning tags.
- Track the retained-sample ratio after length filtering to avoid accidental over-pruning.
- If many samples fail format QA, inspect the original source-specific conversion functions first.

6 Training Configuration and Execution

6.1 Trainer Configuration

6.1.1 Trainer code

Code: SFT training with TRL + Unsloth

```
from trl import SFTTrainer, SFTConfig
trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = dataset,
    eval_dataset = None, # Can set up evaluation!
    args = SFTConfig(
        dataset_text_field = "text",
        per_device_train_batch_size = 6,
        gradient_accumulation_steps = 6, # Use GA to mimic batch size!
        warmup_ratio = 0.03,
        #warmup_steps = 60,
        num_train_epochs = 1, # Set this for 1 full training run.
        #max_steps = 50,
        learning_rate = 2e-4, # Reduce to 2e-5 for long training runs
        logging_steps = 1,
        optim = "adamw_8bit",
        weight_decay = 0.001,
        lr_scheduler_type = "linear",
        seed = 3407,
        save_steps = 200,
        save_total_limit = 1,
        save_strategy = "steps",
        report_to = "wandb", # Can use Weights & Biases
        output_dir = drive_output_path,
    ),
)
```

6.1.2 Key training parameters

Parameter	Typical value	Notes
per_device_train_batch_size	1–128	Depends on VRAM
gradient_accumulation_steps	1–128	Increase effective batch size
learning_rate	1e-5–3e-4	Tune carefully
warmup_steps	1–5000	Stabilizes early training
max_steps / num_train_epochs	task-dependent	Prefer steps for reproducibility
logging_steps	1–20	For monitoring
save_steps	10–500	Checkpoint cadence

Table 4: Common training hyperparameters

6.1.3 Detailed notes on key parameters

- **Learning rate:** If loss is unstable, reduce LR; if convergence is too slow, increase slightly.
- **Effective batch size:** $batch_size \times grad_accum$; bigger is often more stable.
- **Sequence length:** Longer context increases VRAM and compute dramatically; start small.
- **Optimizer:** `adamw_8bit` helps reduce memory usage.

6.2 Train on Responses Only (Recommended)

In most chat fine-tuning settings, you want the model to learn only the assistant responses, while not backpropagating on the user instruction or template tokens. Unsloth provides `train_on_responses_only` to automatically mask labels outside the assistant spans.

Code: Mask labels to train only on assistant responses

```
from unsloth.chat_templates import train_on_responses_only
trainer = train_on_responses_only(
    trainer,
    instruction_part = "<|im_start|>user\n",
    response_part = "<|im_start|>assistant\n<think>",
)
```

Tip: Why this matters

If you train on the entire serialized prompt, the model may waste capacity learning to reproduce system/user content and special tokens, which can hurt instruction-following quality.

6.3 Label Sanity Check (Before/After Training)

To verify that masking worked as expected, you can decode a sample's labels. In many trainers, tokens you do not want to train on are set to `-100` (ignore index). The snippet below replaces those positions with the pad token id so decode can run, and then replaces the visible pad token with spaces for readability.

Code: Decode labels to verify masking

```
tokenizer.decode([tokenizer.pad_token_id if x == -100 else x for x in trainer.
    train_dataset[100]["labels"]]).replace(tokenizer.pad_token, " ")
```

7 Start Training (Most Important)

After the trainer is configured (and optionally masked to responses-only), start training:

Code: Run training

```
trainer_stats = trainer.train()
```

Tip: What is trainer_stats?

`trainer_stats` typically contains training metrics (e.g., loss, steps, runtime). You can print or log it for quick validation.

8 Saving and Deployment

8.1 Saving a 16-bit model

After training, you can push a merged 16-bit model directly to Hugging Face Hub for standard Transformers inference and sharing.

Tip: What this block does

- Reads `HF_TOKEN` securely from Colab Secrets (instead of hard-coding tokens in notebooks).
- Verifies the token via `whoami()` and auto-builds your target repo id.
- Uploads the merged 16-bit checkpoint with tokenizer, so the model can be loaded directly from Hugging Face Hub.

Code: Push merged 16-bit model to Hugging Face Hub

```python
from huggingface_hub import whoami
from google.colab import userdata

try:
    hf_token = userdata.get("HF_TOKEN")
    if not hf_token:
        raise ValueError("HF_TOKEN is not set")
except Exception as e:
    raise RuntimeError("HF_TOKEN was not found in Colab Secrets.") from e

try:
    username = whoami(token=hf_token)["name"]
    repo_id = f"{username}/Qwopus3.5-27B"
except Exception as e:
    raise RuntimeError("Failed to authenticate with Hugging Face.") from e

model.push_to_hub_merged(
    repo_id,
    tokenizer,
    save_method="merged_16bit",
    token=hf_token,
)

print(f"Uploaded to https://huggingface.co/{repo_id}")
```

8.2 Exporting GGUF (optional)

If you want to run the model in `llama.cpp`-compatible stacks, export and publish a GGUF quantized model.

Tip: What this GGUF step does

- Reads `HF_TOKEN` from Colab Secrets and validates Hugging Face identity.
- Exports and uploads multiple GGUF variants (`q4_k_m`, `q8_0`, `bf16`) to one Hub repo.
- Makes the model directly usable in `llama.cpp`-style runtimes with different quality/speed trade-offs.

Warning: Compatibility note

GGUF export depends on the model architecture and your environment. Follow the latest Unsloth / `llama.cpp` guidance for Qwen-family compatibility.

Code: Push GGUF model to Hugging Face Hub

```python
from huggingface_hub import whoami
from google.colab import userdata

try:
    hf_token = userdata.get("HF_TOKEN")
    if not hf_token:
        raise ValueError("HF_TOKEN is not set")
except Exception as e:
    raise RuntimeError("HF_TOKEN was not found in Colab Secrets.") from e

try:
    username = whoami(token=hf_token)["name"]
    repo_id = f"{username}/Qwopus3.5-27B-GGUF"
except Exception as e:
    raise RuntimeError("Failed to authenticate with Hugging Face.") from e

model.push_to_hub_gguf(
    repo_id,
    tokenizer,
    quantization_method=["q4_k_m","q8_0","bf16"],
    token=hf_token,
)

print(f"Uploaded to https://huggingface.co/{repo_id}")
```

9 Common Issues and Optimization Tips

9.1 Typical failure modes

Warning: Failure modes checklist

- Out-of-memory (OOM): reduce sequence length, batch size, or increase grad accumulation; enable gradient checkpointing.
- Loss does not decrease: verify data formatting and the chat template; consider lowering the learning rate.
- Overfitting: reduce steps/epochs, increase data diversity, or enable dropout.

9.2 Hyperparameter tuning workflow

Tip: A practical, reproducible workflow

1. **Stage 1: Validate data + template**
   - Run 50–200 steps and confirm the loss decreases.
   - Spot-check decoded prompts and targets (ensure roles/separators are correct).
2. **Stage 2: Learning-rate sweep**
   - Try `[1e-5, 2e-5, 3e-5]` (expand if needed for your scale).
   - Choose the fastest stable loss decrease (avoid divergence/oscillation).
3. **Stage 3: LoRA rank tuning**
   - Try `[16, 32, 64]`.
   - Balance quality vs. VRAM and training time.
4. **Stage 4: Batch size optimization**
   - Increase batch size as VRAM allows.
   - Use gradient accumulation steps to keep an appropriate effective batch size.

10 Summary and Outlook

10.1 Key takeaways

Summary of the end-to-end workflow

This guide covered the full end-to-end workflow for LLM fine-tuning:

1. **Environment:** Google Colab GPU runtime + Unsloth stack, with optional WandB logging and Google Drive checkpoint persistence.
2. **Model loading:** Qwen3.5-27B is loaded in 4-bit mode to fit single-GPU VRAM constraints while keeping training practical.
3. **Parameter-efficient adaptation:** rank-64 LoRA adapters are attached to key projection modules, so only a small parameter subset is updated.
4. **Data pipeline:** mixed reasoning datasets are normalized into one assistant-target format with Qwen chat template and response-only supervision.
5. **Training and release:** SFT is run with memory-aware hyperparameters, then exported to Hugging Face Hub as merged 16-bit weights and optional GGUF variants.

10.2 Typical metrics

Metric	Typical value / observed range
Base model scale	Qwen3.5-27B
Runtime environment	Google Colab (A100/H100 class GPU)
Model loading mode	4-bit quantized base + LoRA adapters
LoRA rank used in this guide	64
Export targets	merged 16-bit + GGUF (q4_k_m / q8_0 / bf16)

Table 5: Configuration-level metrics aligned with the Qwen3.5-27B Colab workflow

Tip: How to interpret these metrics

For 27B-scale fine-tuning on Colab, absolute runtime and loss values are highly sensitive to GPU type, sequence length, effective batch size, and dataset composition. In practice, compare runs within the same setup rather than relying on a single universal number.

10.3 Next learning steps

1. **Strengthen the current Qwen3.5-27B pipeline first**
   - Build a small, high-quality evaluation set for coding/math/reasoning tasks.
   - Compare `r=16/32/64` and sequence length settings under the same data split.
   - Track inference behavior changes (reasoning length, answer accuracy, and format stability).
2. **Improve data quality and supervision strategy**
   - Add harder long-context and multi-step reasoning samples.
   - Remove noisy or contradictory outputs in mixed-source datasets.
   - Experiment with style constraints for more concise final answers.
3. **Move from SFT to preference alignment**
   - Try DPO with pairwise preference data collected from your target tasks.
   - Evaluate whether alignment improves helpfulness without overextending reasoning traces.
   - Keep SFT checkpoints as stable baselines for A/B comparison.
4. **Deployment and reproducibility**
   - Validate both Hub merged models and GGUF exports with the same prompt suite.
   - Document exact package versions and notebook cells for rerunnable training.
   - Add lightweight serving endpoints (e.g., vLLM/TGI) once offline quality is validated.

10.4 References

Selected references

- Unsloth repository: https://github.com/unslothai/unsloth
- Hugging Face Transformers: https://huggingface.co/docs/transformers
- Qwen documentation: https://github.com/QwenLM/Qwen
- LoRA paper: https://arxiv.org/abs/2106.09685
- WandB docs: https://docs.wandb.ai

10.5 Acknowledgements

Acknowledgements

Thanks to the following open-source projects and communities:

- The Unsloth team for an extremely optimized fine-tuning framework.
- The Hugging Face community for a rich model ecosystem.
- Google Colab for accessible cloud GPU experimentation.
- All dataset contributors and open-source model teams.

Congratulations on completing your LLM fine-tuning journey!

You now know how to adapt a general LLM into a customized AI assistant.
Go build and iterate—looking forward to the great models you train!

This guide is compiled from practical experience and will be continuously updated. Questions and suggestions are welcome.

No one starts as an expert. But every expert was once brave enough to begin.